
Design Specification

Processor Core for the RV32I
(RISC-V 32-Bit Integer Base) Instruction Set

Three-stage pipeline, dual master interface
on AXI4-Lite (Advanced eXtensible Interface, revision 4, Lite subset)

Document	Design Specification — RV32I Processor Core
Document type	Design specification
Subject	RV32I processor core (three-stage pipeline)
Revision	1.2
Date	16 September 2026
Author	Bunea Cosmin-Andrei
Status	Released for review

Contents

1	Introduction	3
1.1	Purpose of the document	3
1.2	Validity of the document	3
1.3	Definitions and terms of abbreviations	3
1.4	Requirements and implementation decisions	4
1.5	Relationship with other documents	4
1.6	Notes on the notation used	4
2	Specifications and system integration	5
3	Overview of overall structure	5
3.1	Execution model	5
3.2	Block structure	6
3.3	Operating modes	6
3.4	Data flow	7
4	Interfaces	7
4.1	Hardware interface	7
4.1.1	Clock and reset	8
4.1.2	Instruction bus interface	9
4.1.3	Data bus interface	9
4.1.4	Interrupt interface	10
4.2	Software interface	11
4.2.1	Register map	11
4.2.2	Register fields	13
4.2.3	Performance counters	13
4.2.4	Programming sequences	14
5	Reuse and reusability	15
6	Design of submodules	16
6.1	Instruction fetch	16
6.1.1	Interface description	16
6.1.2	Functional description	16
6.2	Branch predictor	17
6.2.1	Interface description	17
6.2.2	Functional description	17
6.3	Decode and control	17
6.3.1	Interface description	17
6.3.2	Functional description	17
6.4	Register file and operand forwarding	18
6.4.1	Interface description	18
6.4.2	Functional description	18
6.5	Arithmetic path	18
6.5.1	Interface description	18
6.5.2	Functional description	18
6.6	Branch resolution	19
6.6.1	Interface description	19
6.6.2	Functional description	19

6.7	Load/store unit	20
6.7.1	Interface description	20
6.7.2	Functional description	20
6.8	Exception detection	21
6.8.1	Interface description	21
6.8.2	Functional description	21
6.9	Control and status register file	22
6.9.1	Interface description	22
6.9.2	Functional description	22
6.10	Pipeline control	22
6.10.1	Interface description	22
6.10.2	Functional description	22
6.11	Trap sequencing and interrupt handshake	23
6.11.1	Interface description	23
6.11.2	Functional description	23
6.12	RTL organisation	24
7	List of bugs and workarounds	24
8	Annex	25
8.1	Reset state	25
8.2	Instruction set coverage	26

1 Introduction

1.1 Purpose of the document

This document specifies the design of the RV32I processor core: its architecture, the responsibilities of its functional blocks, the interfaces it presents to the rest of the system, its software-visible register set, and the behaviour a system integrator must rely on.

Where the core is narrower than the instruction set architecture (ISA) it is based on, the difference is stated at the point where it applies.

1.2 Validity of the document

The document is valid for the processor core as currently implemented, comprising the fetch stage, the combined decode and execute stage, the writeback stage, and the two bus master interfaces.

Blocks the core connects to are described only as far as the contract at the boundary between them and the core requires.

1.3 Definitions and terms of abbreviations

Term	Definition
ALU	Arithmetic Logic Unit. The block that performs the arithmetic, logical, shift and comparison operations of the instruction set.
AMBA	Advanced Microcontroller Bus Architecture. The ARM bus interface family that AXI4-Lite belongs to.
AXI4-Lite	The register-access subset of the AMBA Advanced eXtensible Interface, revision 4. One address and one data word per transaction, no bursts, no transaction identifiers.
BHT	Branch History Table. The array holding the recorded direction of a previously executed control transfer.
BTB	Branch Target Buffer. The array holding the recorded target address of a previously executed control transfer.
Bubble	A pipeline slot that carries no instruction. Marked by a cleared valid bit in a pipeline register.
CSR	Control and Status Register. The software-visible register space defined by the RISC-V privileged architecture, accessed by dedicated instructions rather than by load and store.
ISA	Instruction Set Architecture.
M-mode	Machine mode. The highest privilege level of the RISC-V privileged architecture, and the only one implemented by this core.
PC	Program Counter.
RAS	Return Address Stack. A small stack of return addresses used to predict the target of function returns.
RV32I	The 32-bit integer base instruction set of the RISC-V ISA.
WARL	Write Any Values, Reads Legal Values. A register field that accepts any written value without raising an exception but is only required to read back a legal one.
WFI	Wait For Interrupt. The instruction that suspends instruction execution until an interrupt becomes pending.
WPRI	Write Preserve, Read Ignore. A reserved register field that reads as zero and ignores writes.

1.4 Requirements and implementation decisions

The original Siemens briefs are kept in `cpu/docs/brief/`: *RV32I 3-Stage Pipeline CPU* and *Programmable Interrupt Controller with Advanced Scheduling*. They define the project scope. Instruction and CSR semantics are checked against the cited RISC-V manuals, and bus handshakes against the AXI specification. These authored specifications describe the implementation; they are not independent evidence that the implementation is correct.

Details absent from the briefs, including the register map, bus error policy and the sixteen-source claim/EOI interface, are project decisions. The CPU brief’s eight-source interface and the PIC brief’s sixteen-source interface still require a common interface agreement; the implemented contract is stated explicitly in the design specifications.

1.5 Relationship with other documents

Document	Relationship
Verification Specification — RV32I Processor Core	Specifies the verification environment applied to the design described here, and what that environment establishes.
The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, version 20191213	Defines the RV32I base instruction set implemented by the core.
The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, version 20211203	Defines the M-mode privileged subset, the CSR space and trap behaviour that the core implements.
AMBA AXI and ACE Protocol Specification, ARM IHI 0022E	Defines the AXI4-Lite protocol used by both bus master ports.

1.6 Notes on the notation used

- Fixed-width type denotes a signal, a register or a literal value, for example `ARVALID` or `mstatus`.
- Italic type marks emphasis; bold type marks a run-in heading.
- Numeric constants are hexadecimal when prefixed `0x`, otherwise decimal. Bit ranges are written most significant bit first, as `[31:16]`.
- The pipeline stages are referred to as S1 (fetch), S2 (decode and execute) and S3 (writeback) throughout.
- “The core” means the processor core specified by this document. “The system” means whatever the core is integrated into.
- In interface tables, direction is given from the point of view of the core: an output leaves the core.

2 Specifications and system integration

The core is a bus master only. It contains no memory-mapped registers, no memory and no interconnect, and it therefore cannot operate standalone. A system integrating it must supply the following.

Table 2: What the core requires from the system

Resource	Requirement
Instruction memory	An AXI4-Lite slave reachable on the instruction port, holding executable code at the reset vector. The port issues reads only.
Data memory and peripherals	AXI4-Lite slaves reachable on the data port. The core issues no transaction outside what the running program requests, so the address map is entirely a software concern; the core itself decodes no addresses.
Interrupt controller	A block that resolves external interrupt sources, presents one prioritised request together with its source number, consumes the claim and end-of-interrupt pulses the core emits, and reports its full pending set. Section 4.1.4 specifies this interface.
Clock and reset	One clock. One asynchronous, active-low reset, released synchronously or asynchronously; the core contains no clock-domain crossing and no internally generated clock or reset.

The core implements the RV32I base integer instruction set and the M-mode subset of the privileged architecture needed for precise traps and external interrupts. No standard extension is implemented: there is no hardware multiply or divide, no floating point, no atomic instructions and no compressed instruction support. Both bus ports comply with AXI4-Lite and use 32-bit addresses and 32-bit data.

The core is written in Verilog–2005 within the synthesisable subset, but the current target is behavioural simulation; see Section 7.

3 Overview of overall structure

3.1 Execution model

The core executes RV32I in three pipeline stages.

Table 3: Pipeline stages

Stage	Name	Work performed
S1	Fetch	Program counter generation, the instruction bus read, and the branch prediction lookup for the instruction being offered.
S2	Decode and execute	Decode, register read, arithmetic, branch resolution, CSR access, the complete data bus transaction, and every exception and interrupt decision.
S3	Writeback	Writes the result into the register file. No other effect.

Placing decode, execution and the memory access in a single stage means an instruction either completes in full or produces no architectural effect at all. That is what makes traps precise without any recovery mechanism: no half-completed instruction ever exists for a trap to repair. The cost is a long combinational path through decode, the ALU, address formation and the alignment check, which would constrain the achievable clock frequency in a synthesised implementation.

3.2 Block structure

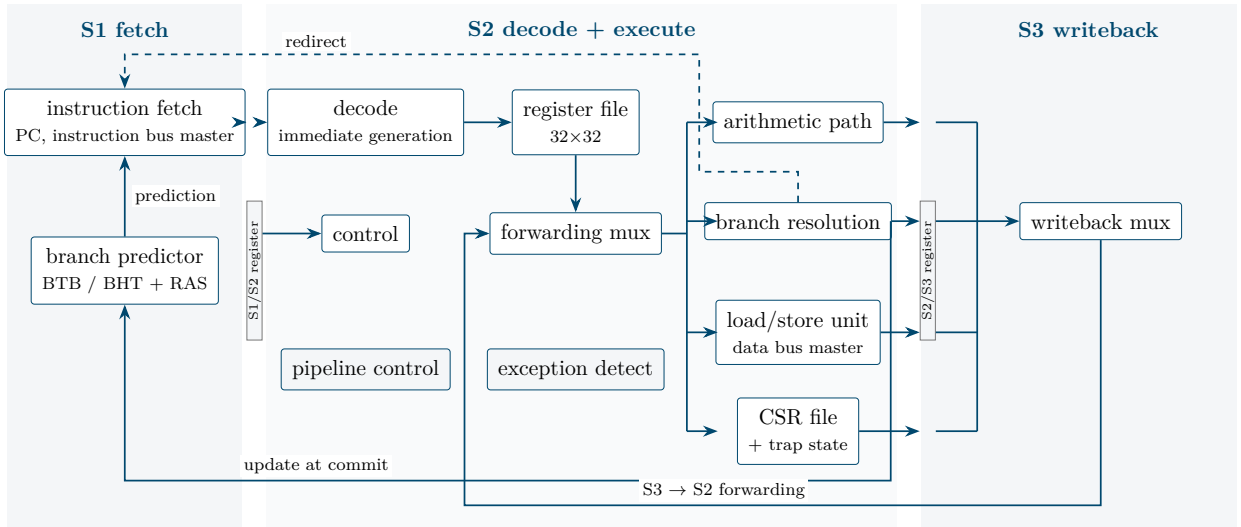


Figure 1: Core block structure. The two vertical bars are the pipeline registers; everything between them belongs to one stage. Instructions travel left to right. Two paths run backwards: the writeback value returns to the operand mux so that an instruction in S2 can use a result that has not yet reached the register file, and the resolved outcome of a control transfer returns to the fetch stage, both as a redirect of the fetch address and as an update of the predictor.

The operand bus drawn as a vertical rail in S2 distributes the two register operands, after forwarding, to all four execution blocks; the result rail on the right collects their results into the pipeline register. Only one of the four produces the value that is finally written back, selected by the writeback mux according to the instruction class.

The blocks divide into two groups, which matters when judging what a change to the instruction set would cost. The register file, the arithmetic path, the load/store unit, the pipeline control and the two bus masters are generic: they are parameterised by width and by operation code, and nothing in them encodes which instruction set is being executed. Decode, immediate generation, branch resolution, exception detection and the CSR file are specific to RV32I and to the M-mode privileged subset; they are where the instruction encoding, the cause numbering and the register addresses live.

3.3 Operating modes

The core has no mode register and no software-selectable modes. It is always in M-mode. Its cycle-by-cycle behaviour is one of four conditions.

Table 4: Operating conditions

Condition	Behaviour
Running	One instruction completes S2 per cycle. Both bus ports may be active simultaneously, each with at most one outstanding transaction.
Stalled	S2 holds its instruction. Two conditions cause this: a data transaction in flight, and a sleeping wait-for-interrupt instruction. S3 receives bubbles for the duration, so nothing commits twice.
Fetch starved	S1 has no instruction to offer. S2 is not stalled; it advances with a bubble. Throughput is lost, but no state is held.
Sleeping	A committed wait-for-interrupt instruction is waiting for a locally enabled interrupt to become pending. Instruction fetch is suspended, so the instruction bus goes quiet.

3.4 Data flow

An instruction is fetched by S1 over the instruction bus, together with the prediction made for it. Both travel to S2 in the pipeline register. In S2 the instruction is decoded, its operands are read from the register file (bypassed from S3 where necessary), and exactly one of the execution blocks produces its result. A load or store instruction additionally runs a complete data bus transaction inside S2, stalling the stage until the response handshake completes. At the end of S2 the instruction either commits — passing its result to S3, updating the predictor, and updating the CSR file if it is a CSR instruction — or is squashed by a trap. S3 writes the committed result into the register file and has no other effect.

Three events resolved in S2 change the fetch path: a trap, a return from trap, and a mispredicted control transfer. All three are delivered to S1 as a single redirect with a target address.

4 Interfaces

4.1 Hardware interface

All ports are synchronous to the single clock input except the reset, which is asynchronously asserted. Every port is digital and unidirectional; the core drives no bidirectional, tri-state or analogue signal.

Table 5: Port summary. Direction is given from the point of view of the core; a width is in bits.

Interface	Signals	Width	Dir.	Purpose
Clock	Clock	1	in	Single clock for all sequential logic
Reset	Reset	1	in	Asynchronous, active low
Instruction bus, read address	Address	32	out	Requests the next instruction; the slave replies on Ready
	Protection type	3	out	
	Valid	1	out	
	Ready	1	in	
Instruction bus, read data	Data	32	in	Returns the instruction word and its response code
	Response code	2	in	
	Valid	1	in	
	Ready	1	out	
Data bus, write address	Address	32	out	Requests a store
	Protection type	3	out	
	Valid	1	out	
	Ready	1	in	
Data bus, write data	Data	32	out	Carries the value and the live byte lanes
	Byte strobes	4	out	
	Valid	1	out	
	Ready	1	in	
Data bus, write response	Response code	2	in	Completes the store
	Valid	1	in	
	Ready	1	out	
Data bus, read address	Address	32	out	Requests a load
	Protection type	3	out	
	Valid	1	out	
	Ready	1	in	
Data bus, read data	Data	32	in	Returns the loaded word and its response code
	Response code	2	in	
	Valid	1	in	
	Ready	1	out	
Interrupt	Request	1	in	One resolved request, its source, and the controller's full pending state
	Source number	4	in	
	Pending set	16	in	
	Enable mask	16	out	The core's enable state, its two handshake pulses, and its trap state
	Claim	1	out	
	End of interrupt	1	out	
	Trap active	1	out	

The sections below give the behaviour of each interface and the timing at which it is exercised.

4.1.1 Clock and reset

All sequential logic is triggered on the rising clock edge. The reset is asynchronously asserted and active low.

Every flip-flop in the core is reset, including the pipeline register payloads and the predictor arrays, which a valid bit would otherwise cover. The rule applied is that no register may leave reset holding an undefined value, so that a two-state and a four-state simulation agree from the first cycle and no bus output or architectural read can ever return an undefined value. The reset state of the architecturally visible elements is listed in the annex.

Bus request outputs are held inactive for as long as reset is asserted, as the bus protocol requires.

4.1.2 Instruction bus interface

An AXI4-Lite master port restricted to the read channels. The core never writes instruction memory, so the write channels do not exist on this port.

Table 6: Instruction port characteristics

Property	Value
Channels	Read address, read data.
Outstanding transactions	At most one.
Address alignment	Always word aligned.
Protection type	Constant: instruction access, privileged, non-secure.
Response handling	An error response does not stall the port. The returned word is delivered to S2 marked as faulted and raises an instruction access fault there, so a wrong-path fetch error is discarded before it can trap.

Back-to-back fetching is supported: the next read address is presented in the same cycle the current read data is accepted, so a slave that answers in one cycle sustains one instruction per cycle.

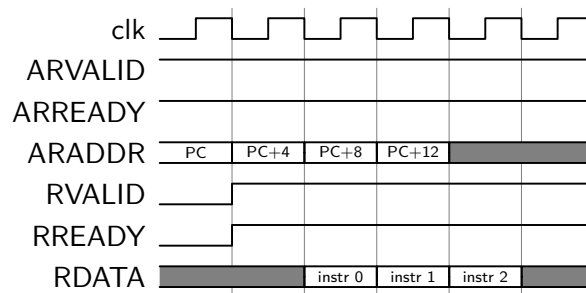


Figure 2: Back-to-back instruction fetch against a slave with no wait states. A new address is presented every cycle while the response for the previous one arrives in parallel, one cycle behind. The sequence to observe is that ARVALID never drops between transactions: the address for the next fetch is issued in the same cycle the current read data is accepted.

4.1.3 Data bus interface

A full AXI4-Lite master port. All five channels are used.

Table 7: Data port characteristics

Property	Value
Channels	All five: write address, write data, write response, read address, read data.
Outstanding transactions	At most one, and never a read and a write at the same time.
Address alignment	Byte addresses are legal. A misaligned access is detected before issue and traps without producing any transaction, so every address that reaches the port is naturally aligned for its size.
Byte enables	Byte and half-word stores are performed with byte strobes, not by read-modify-write. The strobe pattern is one of the seven legal byte, half-word and word shapes.
Protection type	Constant: data access, privileged, non-secure.
Response handling	An error response raises a load or store access fault in S2. The instruction does not commit.

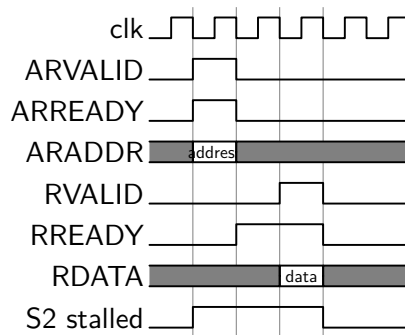


Figure 3: A load against a slave that takes two cycles to answer. The read data channel is only made ready after the address handshake has completed, and the decode and execute stage remains stalled until the data beat is accepted — not until it merely arrives. This is what keeps the transaction indivisible with respect to interrupts.

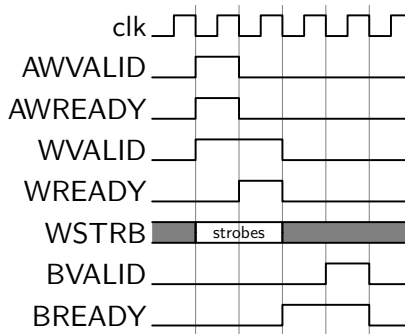


Figure 4: A store where the slave accepts the address one cycle before the data. The write response channel is only made ready once both the address and the data handshakes have completed, which is why it rises later than the address handshake alone would suggest. The two address and data channels are tracked independently and may complete in either order.

4.1.4 Interrupt interface

The core has one external interrupt input, presented as a single prioritised request together with the number of the source that produced it. Resolution between sources is performed outside the core.

Table 8: Interrupt interface

Signal group	Dir.	Description
Request	in	One prioritised interrupt request from the interrupt controller. Level sensitive.
Source number	in	Four bits identifying which of the 16 external sources the request belongs to. Selects the enable bit applied to the request, and the trap cause reported to software.
Pending set	in	The interrupt controller’s complete pending mask, 16 bits. Presented to software as the machine external half of the pending register, so a handler can see the sources queued behind the one being served.
Enable mask	out	The machine external half of the interrupt-enable register, 16 bits, so the external resolver can skip sources this core has masked.
Claim	out	One-cycle pulse in the cycle after the core accepts an interrupt.
End of interrupt	out	One-cycle pulse when a return from trap closes an interrupt level. A return that closes a synchronous exception emits no pulse.
Trap active	out	High while any trap level is open. Provided for observability; the core does not require it to be consumed.

An interrupt is accepted only when all of the following hold in the same cycle: a valid instruction is present in S2, no data transaction is in flight, the global interrupt enable is set, and the enable bit selected by the source number is set. The middle condition is what makes an accepted interrupt land on an instruction boundary and never inside a bus transaction, so the preempted instruction can be re-executed unchanged on return.

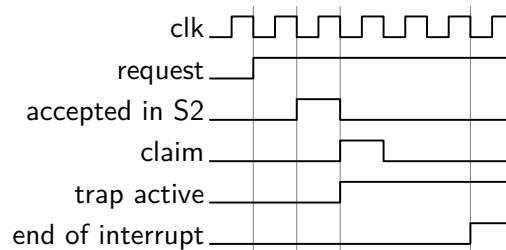


Figure 5: Interrupt claim and return. The request is level sensitive and is held by the controller. The core decides to accept it in S2; the claim pulse is registered and therefore appears one cycle later, at which point the trap-active output rises. The end-of-interrupt pulse follows the return instruction that closes this interrupt level. Both pulses are exactly one cycle wide.

4.2 Software interface

The core exposes no memory-mapped registers. Its entire software interface is the CSR space, accessed by the CSR instructions. Reading an unimplemented CSR, or writing a read-only one, raises an illegal instruction exception rather than a bus error, because the CSR space is internal to the core.

4.2.1 Register map

Table 9: Implemented control and status registers

Address and name	Access / reset	Function
0x300 mstatus	read/write, 0x1800	Global interrupt enable and its saved copy reset to zero; MPP reads machine mode.
0x301 misa	WARL, 0x4000_0100	Reports a 32-bit core implementing the integer base set. Writes are accepted and ignored.
0x304 mie	read/write, 0	Per-source interrupt enables, in bits [31:16]. Bits [15:0] read zero and are not writable.
0x305 mtvec	read/write, 0	Trap vector base and mode.
0x320 —	absent	Counter inhibit is deliberately not implemented; an access raises an illegal instruction exception.
0x323--0x327 mhpmevent3..7	fixed WARL	Event identifiers 1 through 5; writes are ignored.
0x328--0x33F mhpmevent8..31	fixed zero	Unused event selectors; writes are ignored.
0x340 mscratch	read/write, 0	Scratch word for the trap handler.
0x341 mepc	read/write, 0	Trap return address. The two least significant bits always read zero.
0x342 mcause	read/write, 0	Cause of the most recent trap.
0x343 mtval	read/write, 0	Payload associated with the most recent trap.
0x344 mip	read-only, 0	Pending interrupts, bits [31:16], driven by the interrupt controller.
0xB00 mcycle	read/write, 0	Free-running cycle counter, low half.
0xB02 minstret	read/write, 0	Retired instruction counter, low half.
0xB03--0xB07 mhpmcounter3..7	read/write, 0	Performance counters, low halves. See Table 13.
0xB08--0xB1F mhpmcounter8..31	read-only zero	Not provided; read zero and swallow writes.
0xB80, 0xB82--0xB87 upper halves	read/write, 0	Upper halves of the counters above. Address 0xB81 is unimplemented.
0xB88--0xB9F upper halves	read-only zero	Upper halves of the counters that are not provided.
0xF11--0xF13 mvendorid, marchid, mimpid	read-only, 0	Identification registers, all zero.
0xF14 mhartid	read-only	Hart identifier; takes the value of the corresponding parameter.

Any CSR address not listed above is unimplemented. A read or a write to it raises an illegal instruction exception.

4.2.2 Register fields

Table 10: `mstatus` fields. All other bit positions are WPRI: they read zero and ignore writes.

Bits	Field	Meaning
[3]	MIE	Global interrupt enable. Cleared by hardware on trap entry.
[7]	MPIE	The value MIE had before the most recent trap entry. Restored into MIE by a return from trap, which then sets MPIE.
[12:11]	MPP	Previous privilege, hardwired to machine mode.

Table 11: `mtvec` fields

Bits	Field	Meaning
[31:2]	BASE	Trap handler base address, word aligned. Fully writable.
[1:0]	MODE	0 = direct, 1 = vectored. This field is WARL: the reserved encodings 2 and 3 are accepted without trapping but read back as direct mode, so a read never reports a mode the core does not implement.

In direct mode every trap enters at BASE. In vectored mode an interrupt enters at BASE plus four times its cause code, while a synchronous exception still enters at BASE.

Table 12: `mcause` encoding

Bits	Field	Meaning
[31]	Interrupt	Set for an interrupt, clear for a synchronous exception.
[30:5]	—	Zero.
[4:0]	Code	Cause code. Synchronous codes are listed in Table 18; an external interrupt from source n reports code $16 + n$.

4.2.3 Performance counters

Every counter is architecturally 64 bits and is read through a pair of 32-bit CSR addresses. An explicit write takes priority over the implicit increment for that cycle. CSRRW replaces the addressed half; CSRRS and CSRRC apply their mask to its value before the clock edge. The other half remains unchanged, including when an increment would otherwise carry into it. A zero-source CSRRS or CSRRC is a pure read, so counting continues. Counting resumes on the next cycle after a write.

Table 13: Counted events

Counter	Event
<code>mcycle</code>	Every clock cycle.
<code>minstret</code>	Each non-trapping instruction completing S2, in program order. An explicit write to this counter suppresses that cycle's increment.
<code>mhpmcounter3</code>	Mispredicted control transfers.
<code>mhpmcounter4</code>	Cycles in which S2 had no instruction to execute.
<code>mhpmcounter5</code>	Cycles in which a data transaction was in flight.
<code>mhpmcounter6</code>	Trap entries, of either kind.
<code>mhpmcounter7</code>	Cycles spent sleeping in wait-for-interrupt.

The event assignment is fixed in hardware. `mhpmevent3..7` read the implementation-defined identifiers 1 through 5, in the order shown above. They accept writes but retain their fixed values.

Identifier zero denotes no event and is returned by the unused selectors `mhpmevent8..31`. There is no counter inhibit register; enabled event counting is continuous except when an explicit counter write takes priority.

An instruction reading `minstret` observes the count before its own completion. Counting at the S2 completion boundary places preceding instructions before the CSR access and following instructions after it, independently of the S3 register-file write.

4.2.4 Programming sequences

The procedures below follow from the register behaviour above and from the trap sequencing described in Section 6.11.

Preparing for traps. Write `mtvec` before enabling anything that can trap. Until it is written it reads zero, so a trap taken before that point enters at address zero. In vectored mode the table must have an entry for every cause that can occur: an external interrupt from source n enters at $\text{BASE} + 4 \times (16 + n)$, so the sixteen external entries occupy the words at offset 64 to 124, while every synchronous exception enters at BASE itself.

Enabling an external interrupt. Three independent enables must all permit the source, and they are checked in different places:

1. the enable inside the interrupt controller, which decides whether the source is resolved at all;
2. the corresponding bit of `mie`, which the core also exports to the controller so that its resolver skips sources this core has masked;
3. `mstatus.MIE`, the global enable.

The order of the three writes does not create a spurious trap in either direction, because a request that passes only some of them is simply not accepted. Setting the global enable last is nonetheless the safer habit, since it is the point after which a handler can actually be entered.

Inside a handler. On entry the core has already recorded `mepc`, `mcause` and `mtval`, copied `mstatus.MIE` into `MPIE` and cleared `MIE`. A handler therefore starts with interrupts globally disabled, and **must set MIE itself if it wants to be preemptable**. Bit 31 of `mcause` distinguishes an interrupt from a synchronous exception; for an interrupt the low bits give $16 + n$, for an exception they give the cause code from Table 18.

Leaving a handler. Return with the return-from-trap instruction, not with a jump. The instruction is what restores `MIE` from `MPIE` and what closes the trap level; it is also what emits the end-of-interrupt pulse to the controller when the level being closed was an interrupt. A handler that returns by jumping leaves the level open, and the controller will hold that source in service indefinitely.

Reading a 64-bit counter. Each counter is read through two 32-bit addresses and there is no atomic snapshot, so the low half can wrap between the two reads. The standard sequence applies: read the high half, read the low half, read the high half again, and repeat if the two high reads differ.

Reading a register without writing it. A set or clear operation with a zero source register — or a zero immediate in the immediate forms — is architecturally a pure read and is treated as one here: it does not count as a write, so it does not raise an exception on a read-only register. This is the way to sample `mip` or the identification registers. A read/write operation always counts as a write, even when its destination is the zero register, and will therefore raise an exception on a read-only register.

Waiting for an interrupt. The wait instruction suspends execution until a source enabled in `mie` becomes pending. The global enable is deliberately not part of the wake condition. The consequence is two different behaviours: with the global enable set, the wake is taken as a normal interrupt and `mepc` points at the instruction *after* the wait, so the return does not re-enter it; with the global enable clear, the wait simply completes as a no-operation and execution continues in line, with no trap and no handler entry.

5 Reuse and reusability

No component of the core is sourced from elsewhere: every block is written for this design, so there is no third-party or externally specified element whose own documentation would have to be consulted.

The core is written as a self-contained, parameterised block with no shared or global state, and is instantiable more than once in the same system. Its submodules are individually reusable in the sense that each has a narrow interface and no hidden dependency on the others, but they are not intended as standalone deliverables.

Table 14: Parameters

Parameter	Default	Effect
Reset vector	0x0000_0000	The first address fetched after reset.
Hart identifier	0	The value read from <code>mhartid</code> . Lets software running on several instances distinguish them, which is what makes a multi-core configuration usable.
Predictor entries	128	Depth of the branch target and history arrays. Must be a power of two. Affects prediction accuracy and register count only; no architectural behaviour depends on it.
Return stack depth	8	Depth of the return address stack. Zero removes the stack entirely, in which case returns are predicted from the branch target array like any other control transfer. Accuracy only.

Both prediction parameters affect accuracy and never correctness, so they may be changed without revalidating architectural behaviour. A resolved control transfer always computes its own target and redirects the fetch stage if the prediction was wrong, so even a predictor that is always wrong costs cycles and nothing else.

For a multi-instance configuration the integrator must give each instance a distinct hart identifier and provide separate bus paths or arbitration; the core arbitrates nothing itself. Since every instance is in-order with blocking memory operations, no reservation or atomic-access mechanism is required for software running on separate instances to synchronise through shared memory.

6 Design of submodules

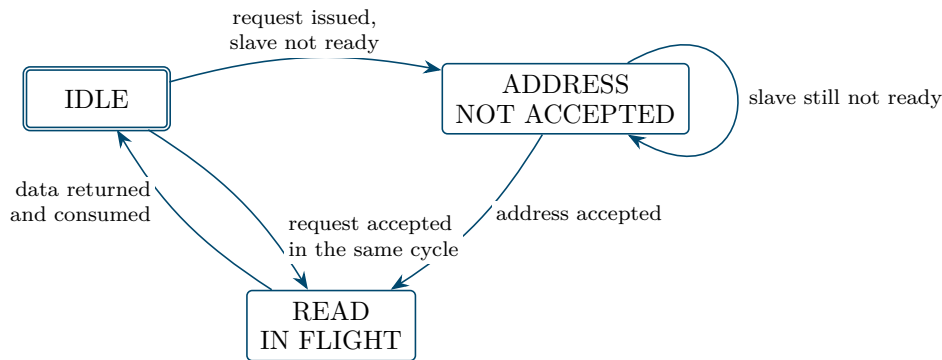
6.1 Instruction fetch

6.1.1 Interface description

Inputs: the redirect request and target from S2, and the prediction produced for the address currently being offered. Outputs: the instruction bus read channels, the address to be looked up in the predictor, and the instruction handed to S2 together with its address, its prediction and a flag marking a faulted fetch.

6.1.2 Functional description

The block owns the program counter and keeps at most one instruction read in flight. Its behaviour is described by three independent conditions rather than by a state register: an address request issued but not yet accepted, a read accepted and its data still owed, and an in-flight read known to be on the wrong path.



The discard condition is independent of the three above.

It is armed by a redirect arriving while a read is outstanding and cleared when the unwanted data word is swallowed. It cannot be a fourth state: an issued read cannot be cancelled, only its result ignored.

Figure 6: Behaviour of the instruction read slot. The double outline marks the condition after reset. None of the three names exists as a state register; they are produced by three independent flags, which is why the discard case can overlap either busy condition.

Two mechanisms deserve attention because they are not obvious from the interface.

Discarding a wrong-path fetch. An AXI read cannot be withdrawn once its address has been presented. When S2 redirects the fetch path while a read is outstanding, the block therefore marks that read for discard, accepts its data word, drops it, and restarts from the redirect target.

Holding register. If S2 is stalled when an instruction arrives, the instruction is parked in a one-entry holding register and no new fetch is issued until it drains. S1 can therefore never run more than one instruction ahead of S2. The same mechanism is what silences the instruction bus during wait-for-interrupt: once the holding register is full, no further address is issued.

An error response on a fetch does not stall the block. The word is delivered with a fault marker and traps in S2, which means a fetch error on a wrong-path instruction is flushed before it can do any harm.

6.2 Branch predictor

6.2.1 Interface description

A read port used by S1, taking the address being fetched and returning a predicted direction and target; and a write port used by S2 at instruction commit, carrying the resolved direction and target plus three hints that classify the committing instruction as a call, a return, or neither.

6.2.2 Functional description

The predictor combines three structures.

Table 15: Predictor structures

Structure	Function
Direction history	One bit per entry, holding the last resolved outcome of the control transfer at that entry. That bit is the prediction for the next encounter.
Target buffer	One 32-bit target per entry. It is what makes prediction possible at all: at fetch time the instruction has not been decoded, so a direction bit alone cannot say where to jump.
Return address stack	A separate stack of return addresses. A return usually has a different target from the last time it executed, so the stored target predicts it badly; the address to return to, on the other hand, is known exactly at the call.

The arrays are direct mapped and indexed by the low address bits, with the remaining address bits stored as a tag. A lookup predicts taken only on a tag match with the direction bit set; a miss predicts not taken, which is the only possible prediction since there is no stored target to jump to. An entry recorded as a return is predicted from the top of the return stack instead of from its stored target, falling back to the stored target if the stack is empty.

The stack is maintained at the same commit point as the other two arrays, so no predictor state is ever speculative. Calls and returns are identified from the destination and source register numbers, following the hint encoding defined by the instruction set. An instruction that links and also returns is treated by that encoding as two different cases: when the two register numbers are equal it is a call only and pushes; when they differ it is a return followed by a call, and the stack replaces its top entry rather than changing its depth.

Stack overflow wraps and discards the oldest entry, and a pop on an empty stack falls back to the stored target. Both cost prediction accuracy only. Correctness does not depend on the predictor at any point: the real direction and target are always computed in S2, and a wrong prediction is corrected by a redirect.

6.3 Decode and control

6.3.1 Interface description

Input: the instruction word from the pipeline register. Outputs: the extracted instruction fields, the immediate value assembled for the instruction format, and the control signals that steer the rest of S2.

6.3.2 Functional description

Field extraction is pure wiring. The immediate generator assembles the immediate for the five instruction formats from the same instruction bits.

The control decoder is strict: it validates the encoding rather than assuming it. An unknown opcode, a reserved function field, an undefined load or store width, an undefined branch condition, or a system instruction that does not match one of the exact defined encodings all raise an illegal instruction indication.

An illegal instruction has no architectural effect, but that is guaranteed downstream rather than by the decoder withholding its outputs. An unknown opcode leaves every control line at its inactive default; a defined opcode with a reserved function field still asserts the control lines of its class, and the illegal indication is raised alongside them. What makes the instruction harmless in both cases is that the exception it raises squashes it before it can commit, and that the memory issue is separately conditioned on the instruction being legal, so no bus transaction is started either.

Two decoding decisions are worth stating:

- The fence instruction is decoded as a no-operation. The core is a single in-order hart with blocking memory accesses, so there is nothing to order. The instruction-fence variant is not implemented and is decoded as illegal.
- A CSR instruction produces a register write whose source is the CSR read value. The read value is substituted into the arithmetic result path at the end of S2, so no separate writeback source is needed.

6.4 Register file and operand forwarding

6.4.1 Interface description

Two combinational read ports addressed from S2, and one write port driven from S3.

6.4.2 Functional description

Thirty-two 32-bit general purpose registers. Register zero is hardwired: it reads zero on both ports regardless of the array contents, and a write to it is discarded. All registers are cleared by reset.

The register file write happens on the clock edge that ends S3, so an instruction in S2 that reads a register the S3 instruction is writing would otherwise read a stale value. A bypass compares the S3 destination against both S2 source registers, excluding register zero, and substitutes the writeback value for the register file output on a match.

This bypass is also what removes the classic load-use penalty. In a three-stage pipeline a load's data has already reached S3 by the time the consuming instruction enters S2, so a single backward path covers the case and no stall cycle is needed.

6.5 Arithmetic path

6.5.1 Interface description

Inputs: the two operands, the operand-select control, the operation class from the decoder, and the instruction function fields. Output: the result.

6.5.2 Functional description

The block resolves the operation in two levels. The decoder produces a coarse operation class — address arithmetic, register or immediate arithmetic, immediate pass-through, or program-counter-relative addition — and the arithmetic path refines that class into a concrete operation using the instruction's function fields. This is what allows the decoder to remain independent of the arithmetic encoding.

Table 16: Operation decode. The class comes from the instruction decoder; the function fields come from the instruction. A class the decoder cannot produce resolves to addition.

Class	funct3	funct7	Operation
Address arithmetic	—	—	Add
Immediate pass-through	—	—	Pass the second operand
Program-counter relative	—	—	Add
Register or immediate arithmetic	000	0100000	Subtract, register form only
	000	other	Add
	001	—	Shift left logical
	010	—	Set if less than, signed
	011	—	Set if less than, unsigned
	100	—	Exclusive or
	101	0100000	Shift right arithmetic
	101	other	Shift right logical
	110	—	Or
	111	—	And

The subtract entry is conditional on the register form because the immediate form has no subtract encoding; with an immediate operand the same function fields select an addition. Shift amounts use the low five bits of the second operand, so a larger value wraps as the ISA requires.

The second operand is selected between the second source register and the immediate. For program-counter-relative addressing the first operand is the instruction address rather than a register.

6.6 Branch resolution

6.6.1 Interface description

Inputs: the instruction address, the immediate, both register operands after forwarding, and the control indication that this is a branch or a jump. Outputs: the resolved target address and the resolved direction.

6.6.2 Functional description

The target is the instruction address plus the immediate, except for the register-indirect jump, where it is the first source register plus the immediate with the least significant bit forced to zero, as the ISA requires.

The direction is the comparison selected by the instruction’s function field for a conditional branch, and unconditionally taken for a jump.

Table 17: Branch condition decode

funct3	Taken when
000	the operands are equal
001	the operands differ
100	the first is less than the second, signed
101	the first is greater than or equal to the second, signed
110	the first is less than the second, unsigned
111	the first is greater than or equal to the second, unsigned

The two encodings absent from the table are undefined for a branch and are rejected by the decoder as illegal instructions, so they never reach this block. It is purely combinational and produces the architecturally correct outcome, against which the prediction made in S1 is compared.

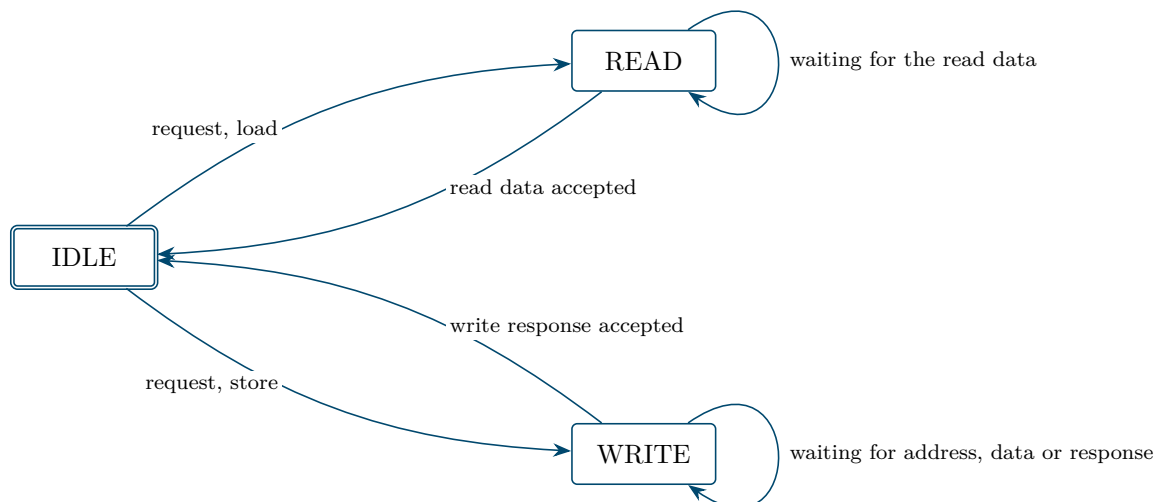
6.7 Load/store unit

6.7.1 Interface description

Inputs from S2: a request, the direction, the access size, the address and the store data. Outputs to S2: a busy indication that stalls the stage, a completion indication, an error indication, an active indication that blocks interrupt acceptance, and the extended load result. Externally, the data bus master port.

6.7.2 Functional description

The unit contains the only explicitly coded state machine in the core.



Reset condition: IDLE. One transaction at a time.
The unit returns to IDLE when the response handshake completes, not when the data arrives.

Figure 7: Load/store state machine. The two self-loops are the normal case, not an exception: a slave may insert any number of wait states. The state alone does not describe progress within a transaction; that is tracked separately, as described below.

The state does not record how far a transaction has progressed, because a write uses three independent channels — address, data and response — and the address may be accepted before the data or after it. Three per-channel trackers record which handshakes have completed. The response channel is only made ready once the channels it depends on have completed: for a read, after the address handshake; for a write, after both the address and the data handshakes. The transaction is finished, and the stage released, only when the response handshake itself completes.

The address, write data and byte strobes are captured when the transaction starts and driven from that capture thereafter. This is required because the forwarded operands are only valid in the first cycle — S3 receives bubbles while S2 is stalled — and because AXI requires a stable payload while a request is waiting to be accepted.

Byte lanes. A byte or half-word store replicates the data across the bus and selects the live lanes with the byte strobes; no read-modify-write is performed. A load shifts the returned word by the address offset and then sign- or zero-extends according to the instruction, giving the five defined load forms.

Errors and alignment. A misaligned access never reaches the bus: it is detected before issue and trapped. An error response on the bus is reported to the exception logic, which raises a load or store access fault.

6.8 Exception detection

6.8.1 Interface description

Inputs: an indication that the instruction in S2 is real and not preempted, the fetch fault marker, the illegal indications from the decoder and the CSR file, the environment call and breakpoint indications, the resolved control transfer, and the state of the data access. Outputs: an exception indication, the cause code, the associated value, and a signal that blocks the bus issue for a misaligned access.

6.8.2 Functional description

All synchronous exceptions are detected in S2 and resolved by a single priority chain, so exactly one cause is reported and the cause list exists in one place. The chain also selects the value recorded alongside the cause.

Table 18: Synchronous causes, in the order the priority chain evaluates them

Order	Cause	Recorded value
1	Instruction access fault (1)	The instruction address
2	Illegal instruction (2)	The instruction word
3	Breakpoint (3)	The instruction address
4	Environment call from M-mode (11)	Zero
5	Instruction address misaligned (0)	The computed target address
6	Load address misaligned (4)	The effective address
7	Store address misaligned (6)	The effective address
8	Load access fault (5)	The effective address
9	Store access fault (7)	The effective address

The numbers in parentheses are the architectural cause codes. The evaluation order is deliberately not the numeric order: it places the conditions that invalidate the instruction itself — a faulted fetch, an illegal encoding — ahead of conditions computed from that instruction’s operands, so that a corrupted instruction is never reported as, for example, a misaligned access derived from its own decoded fields.

Alignment is checked from the effective address before the access is issued, and only before issue: once a transaction is out, its address was already accepted as legal, and the arithmetic result feeding the check may change during the stall.

Interrupts do not pass through this block. They are evaluated at the top level before execution and suppress the instruction instead.

6.9 Control and status register file

6.9.1 Interface description

A CSR access port from S2 carrying the address, the write data, the operation and separate read and write enables, returning the read value and an illegal access indication. Separate ports for trap entry and return, for the interrupt enable and pending exchange with the interrupt controller, for the resulting handler address and return address, and for the counted events.

6.9.2 Functional description

The block holds the architectural state listed in Section 4.2 and performs the read-modify-write defined for the three CSR operations. Only the fields that exist are stored; reserved positions are masked before the value reaches the register rather than being stored and read back, so a later revision can define one of them without breaking software that happened to write a one there.

Address checking distinguishes four access classes, and confusing them is the classic CSR defect, so they are stated explicitly:

Table 19: Control and status register access classes

Class	Rule
Read/write	Reads and writes both take effect.
Read-only	A write raises an illegal instruction exception and the value is unchanged. Trapping is required: it is what lets software discover the mistake instead of believing the write succeeded.
WARL	A write is accepted without trapping, but only the legal part of the value is retained.
Tied off	The register exists but is hardwired to zero. Reads return zero and writes are swallowed; neither traps.
Absent	Any access raises an illegal instruction exception.

A set or clear operation with a zero source is architecturally a pure read and does not count as a write, so it does not trap on a read-only register.

Trap entry is atomic with respect to software: it records the return address, the cause and the associated value, copies the global interrupt enable into its saved copy, and clears the enable, all on the same clock edge. Trap entry takes precedence over a software write in the same cycle, so a trapping instruction never commits its own CSR write. A return from trap restores the enable from its saved copy and sets the saved copy.

6.10 Pipeline control

6.10.1 Interface description

Inputs: whether S1 is offering an instruction, whether a data transaction is in flight, whether a wait-for-interrupt is sleeping, the three redirect requests from S2, and the register numbers needed for the forwarding comparison. Outputs: per-stage advance, bubble, squash and redirect commands, and the two forwarding selects.

6.10.2 Functional description

All stall, flush and forwarding decisions are taken in this one block; no stage manages its own progress. The entire control is a small set of combinational equations, which means there is no state machine driving the pipeline and therefore no invalid pipeline state to reach.

Table 20: Stall and flush sources

Category	Sources
Stall (two)	A data transaction in flight, and a sleeping wait-for-interrupt. An instruction fetch that is not ready is deliberately <i>not</i> a stall: S2 advances with a bubble instead, which costs the same throughput and removes a term from the advance condition.
Flush (three)	A mispredicted control transfer, trap entry, and return from trap. All three produce the same action — request a redirect and load a bubble into the pipeline register — so there is one mechanism rather than three.

Where a stall and a flush are requested together, the flush wins, because it invalidates the very instruction the stall was waiting for. The rule has a second half that comes from the same term: every flush condition is qualified by the stage advance, so a trap cannot leave S2 while a data transaction is still outstanding. The bus is therefore never left with a transaction whose response nobody will accept.

Both pipeline registers carry a valid bit. A cleared valid bit is what makes a bubble safe: nothing downstream examines the payload, so a flush produces a guaranteed no-operation whatever the payload holds. The alternative — injecting an encoded no-operation — would require the flush path to drive 32 correct bits and every downstream decoder to interpret them.

6.11 Trap sequencing and interrupt handshake

6.11.1 Interface description

Internal to the top level. It consumes the interrupt request and source number, the exception indication from the exception logic, and the return instruction indication; it drives the CSR trap ports, the redirect target, and the claim, end-of-interrupt and trap-active outputs.

6.11.2 Functional description

Every architectural decision is taken in S2 in a fixed order:

interrupt > synchronous exception > return from trap > mispredicted control transfer

The interrupt is evaluated first, and specifically before the instruction issues any data transaction, so that a preempted instruction can be re-executed identically after the handler returns. An exception outranks a misprediction because a faulting instruction has no correct path left to follow.

The recorded return address differs by trap kind, as the privileged architecture requires: an exception records the offending instruction, so that a handler that repairs the fault can retry it, while an interrupt records the instruction to resume from. A wait-for-interrupt instruction woken by an interrupt is a special case of the latter: it records the following instruction, so the return does not re-enter the sleep.

Open trap levels. The core keeps one bit per currently open trap level, on a stack sixteen deep, recording whether that level was an interrupt. Trap entry pushes; a return pops, and emits the end-of-interrupt pulse only if the popped bit says the level was an interrupt.

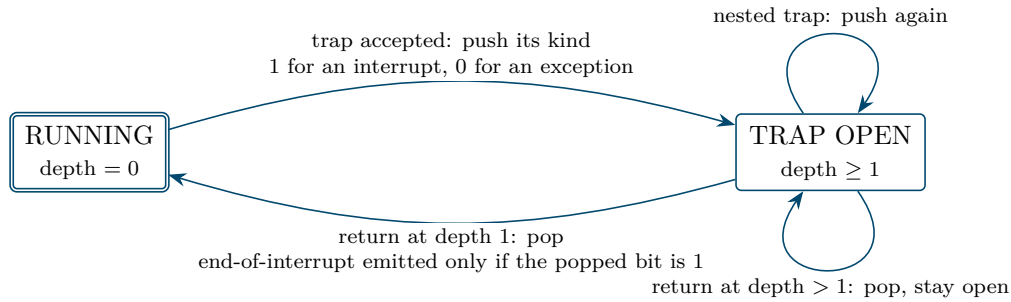


Figure 8: Trap level tracking. The two states describe only whether any trap is open; the information that matters is carried by the stack of kind bits, one per open level. The important transitions are the two self-loops, which are what distinguish this from a single in-progress flag.

A single in-progress flag would be insufficient, because the same return instruction closes both kinds of trap and both kinds can be open at once. Two nested interrupt handlers would send one end-of-interrupt pulse instead of two, leaving the outer level open in the controller with its source permanently in service; and a synchronous exception taken inside an interrupt handler would send an end-of-interrupt on its own return, releasing the interrupt while its handler was still running. One bit per level removes both cases without any additional logic: the kind of the trap being closed is the bit popped from the stack.

The stack is sixteen levels deep, which is also the number of external interrupt sources the interface can express. Beyond sixteen open levels the depth counter saturates and the oldest recorded level is shifted out of the stack, so the end-of-interrupt sequence is no longer guaranteed to be correct. Software must keep the combined number of open interrupt and synchronous exception levels at or below sixteen. Limiting interrupt nesting alone is insufficient when an exception occurs inside a handler. The core does not prevent overflow.

6.12 RTL organisation

The RTL is grouped by function: interfaces and constants, combinational decisions, state updates, and outputs. Each `always` process assigns one state signal or one array; loop indices are local iteration variables. Priority conditions are explicit, and registers have one procedural owner. Existing parameters retain their function. Architectural widths fixed by RV32I or the sixteen-source PIC contract remain fixed rather than exposing unsupported configurations. RTL files contain no ``timescale`; time units belong to the testbenches or simulator command line.

7 List of bugs and workarounds

No functional defect is currently open against the core. The entries below are implementation limitations and deliberate restrictions that a system integrator or a software author needs to know about.

Table 21: Known limitations

Item	Description and consequence
No synthesis result	The core is written within the synthesisable subset, but no synthesis or timing run has been performed. The long combinational path through S2 noted in Section 3 is the expected frequency limiter and has not been measured.
Instruction fence not implemented	The ordinary fence is a no-operation, which is correct for a single in-order hart. The instruction-fence variant is decoded as an illegal instruction. Software that modifies its own instruction stream is therefore not supported.
Counter inhibit absent	All counters run at all times and cannot be stopped. The privileged architecture permits this; software must not assume it can freeze a counter before reading a pair of halves.
Performance counters 8 and above	Not provided. They read zero and swallow writes rather than trapping, as the architecture requires of counters an implementation does not provide. The event selectors are likewise hardwired to zero, so the event assignment in Table 13 cannot be changed by software.
No unaligned access support	Every misaligned load or store traps. This is architecturally permitted, but software or a runtime library must emulate unaligned access if it is needed.
Interrupt source count fixed at 16	The source number input is four bits wide and the enable and pending masks are sixteen bits, so the interface cannot express more than sixteen external sources.
Protection type outputs are constant	Both ports drive a fixed protection encoding. A system that uses the protection field to distinguish privilege levels cannot do so with this core.
Interface deviation from the original requirement	An early requirement described the interrupt interface as a per-source request vector with a three-bit source identifier. The implemented interface is the one specified in Section 4.1.4: a single prioritised request with a four-bit source number, plus an end-of-interrupt pulse that the original requirement did not mention. Sixteen sources cannot be identified in three bits, and the end-of-interrupt pulse is required for the external controller to unwind nested interrupts correctly. This is a difference between two requirement sets rather than a defect, and needs a decision before the interface is frozen.

8 Annex

8.1 Reset state

Reset is asynchronous and active low. Every flip-flop in the core is reset, including the pipeline register payloads and the predictor arrays, which a valid bit would otherwise cover. The rule applied is that no register may leave reset holding an undefined value, so that a two-state and a four-state simulation agree from the first cycle and no bus output or architectural read can ever return an undefined value.

Table 22: Reset state of the architecturally visible elements

Element	Reset value
Program counter	The reset vector parameter
General purpose registers	All zero
<code>mstatus</code>	Interrupts globally disabled
<code>mie</code> , <code>mtvec</code> , <code>mscratch</code> , <code>mepc</code> , <code>mcause</code> , <code>mtval</code>	Zero
<code>misa</code>	Fixed identification value
<code>mhartid</code>	The hart identifier parameter
All counters	Zero
Branch predictor	All entries invalid; return stack empty
Pipeline registers	Invalid, payload set to a no-operation and the reset vector
Load/store unit	Idle, no transaction outstanding
Trap level stack	Empty; trap-active output low
Bus request outputs	Inactive while reset is asserted

8.2 Instruction set coverage

Table 23: Implemented instruction groups

Group	Instructions
Upper immediate	Load upper immediate; add upper immediate to program counter
Jumps	Jump and link; jump and link register
Branches	Equal, not equal, less than, greater or equal, and the two unsigned forms
Loads	Byte, half-word, word, and the two unsigned forms
Stores	Byte, half-word, word
Register-immediate	Add, set less than (signed and unsigned), exclusive or, or, and, and the three shifts
Register-register	The above plus subtract
Ordering	Fence (executed as a no-operation)
System	Environment call, breakpoint, return from trap, wait for interrupt
CSR access	Read/write, read/set, read/clear, and their immediate forms

No instruction outside this list is implemented. Any other encoding raises an illegal instruction exception.